2025-2026 第 2 学期

操作系统实验报告

班　　级：软件 2024-03 班

学　　号：2024112639

姓　　名：毛露涵

专　　业：软件工程

任课教师：李洋

二零二六年六月

目 录

实验 1　观察 Linux 的行为 1
1.1 实验目的 1
1.2 实验内容 1
1.2.1 设计思想及算法流程 1
1.2.2 源程序清单 1
1.2.3 运行结果及解释 6
1.3 实验总结 7
实验 2　软中断通信实验 8
2.1 实验目的 8
2.2 实验内容 8
2.2.1 设计思想及算法流程 8
2.2.2 源程序清单 9
2.2.3 运行结果及解释 11
2.3 实验总结 12
实验 3　线程同步实验 13
3.1 实验目的 13
3.2 实验内容 13
3.2.1 设计思想及算法流程 13
3.2.2 源程序清单 14
3.2.3 运行结果及解释 19
3.3 实验总结 22
实验 4　进程调度实验 23
4.1 实验目的 23
4.2 实验内容 23
4.2.1 设计思想及算法流程 23
4.2.2 源程序清单 25
4.2.3 运行结果及解释 35
4.3 实验总结 37
实验 5　内存管理实验 38

5.1 实验目的 38
5.2 实验内容 38
5.2.1 设计思想及算法流程 38
5.2.2 源程序清单 40
5.2.3 运行结果及解释 47
5.3 实验总结 51
实验 6　系统调用 52
6.1 实验目的 52
6.2 实验内容 52
6.2.1 设计思想及算法流程 52
6.2.2 源程序清单 55
6.2.3 运行结果及解释 56
6.3 实验总结 57
实验 7　系统缺页次数统计 58
7.1 实验目的 58
7.2 实验内容 58
7.2.1 设计思想及算法流程 58
7.2.2 源程序清单 60
7.2.3 运行结果及解释 62
7.3 实验总结 66

实验 1　观察 Linux 的行为

1.1 实验目的

通过本实验，了解 Linux 系统的组织结构，观察各种系统状态信息，并熟悉这些结构与信息的读取。

1.2 实验内容

实验内容：编写程序打印出如下内容（上交的实验 1 名为: test1）

1. CPU 类型及型号、Linux 内核版本号
2. 系统最近一次启动以来经历的时间（以格式:xx 时 xx 分 xx 秒输出）
3. CPU 在三个状态的时间花费（用户态、内核态、空闲状态）
4. 内存的使用情况（已使用、可用）、平均负载
5. 系统启动以来所创建的进程数、上下文转换次数

1.2.1 设计思想及算法流程

程序按照功能划分为三个模块。

1. func_A()：读取/proc/cpuinfo 与/proc/version，提取 CPU 型号并输出内核版本号。
2. func_B()：读取/proc/uptime 与/proc/stat，将系统运行秒数转换为“时-分-秒”格式，并统计用户态、内核态、空闲态时间，以及进程创建数和上下文切换次数。
3. func_C()：读取/proc/meminfo 与/proc/loadavg，计算已使用内存与可用内存，并输出 1 分钟、5 分钟、15 分钟平均负载。

主函数中根据命令行参数进行分发：无参数时执行 func_A()；参数为-s 时执行 func_B()；参数为-l 时执行 func_C()。

1.2.2 源程序清单

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>

void func_A() {
    FILE *cpuinfo = fopen("/proc/cpuinfo", "r");
    FILE *version = fopen("/proc/version", "r");
    char line[2048];
```

```
    int found_model = 0;

    if (cpuinfo == NULL || version == NULL) {
        printf("打开 /proc 文件失败。\n");
        if (cpuinfo) fclose(cpuinfo);
        if (version) fclose(version);
        return;
    }

    printf("========== CPU 类型及型号 ==========\n");
    while (fgets(line, sizeof(line), cpuinfo) != NULL) {
        if (strncmp(line, "model name", 10) == 0) {
            char *p = strchr(line, ':');
            if (p != NULL) {
                printf("%s", p + 2);
                found_model = 1;
                break;
            }
        }
    }
    if (!found_model) {
        printf("未找到 CPU 型号信息。\n");
    }

    printf("\n========== Linux 内核版本号 ==========\n");
    int ch;
    while ((ch = fgetc(version)) != EOF) {
        putchar(ch);
    }
    printf("\n");

    fclose(cpuinfo);
    fclose(version);
}
```

```
void func_B() {
    FILE *uptime = fopen("/proc/uptime", "r");
    FILE *stat = fopen("/proc/stat", "r");

    if (uptime == NULL || stat == NULL) {
        printf("打开 /proc 文件失败。\n");
        if (uptime) fclose(uptime);
        if (stat) fclose(stat);
        return;
    }

    double up_seconds, idle_seconds;
    if (fscanf(uptime, "%lf %lf", &up_seconds, &idle_seconds) == 2) {
        int total = (int)up_seconds;
        int hour = total / 3600;
        int minute = (total % 3600) / 60;
        int second = total % 60;

        printf("========== 系统最近一次启动以来经历的时间 ==========\n");
        printf("%d 时%d 分%d 秒\n\n", hour, minute, second);
    } else {
        printf("读取 uptime 失败。\n\n");
    }

    char line[2048];
    unsigned long long user = 0, nice = 0, system = 0, idle = 0;
    unsigned long long processes = 0, ctxt = 0;

    while (fgets(line, sizeof(line), stat) != NULL) {
        if (strncmp(line, "cpu ", 4) == 0) {
            sscanf(line, "cpu %llu %llu %llu %llu", &user, &nice, &system, &idle);
        } else if (strncmp(line, "processes ", 10) == 0) {
            sscanf(line, "processes %llu", &processes);
```

```
        } else if (strncmp(line, "ctxt ", 5) == 0) {
            sscanf(line, "ctxt %llu", &ctxt);
        }
    }

    printf("========== CPU 三种状态时间花费 ==========\n");
    printf("用户态时间: %llu\n", user);
    printf("内核态时间: %llu\n", system);
    printf("空闲态时间: %llu\n\n", idle);

    printf("========== 系统启动以来的进程与上下文切换信息 ==========\n");
    printf("系统创建的进程数: %llu\n", processes);
    printf("上下文转换次数: %llu\n", ctxt);

    fclose(uptime);
    fclose(stat);
}

void func_C() {
    FILE *meminfo = fopen("/proc/meminfo", "r");
    FILE *loadavg = fopen("/proc/loadavg", "r");

    if (meminfo == NULL || loadavg == NULL) {
        printf("打开 /proc 文件失败。\n");
        if (meminfo) fclose(meminfo);
        if (loadavg) fclose(loadavg);
        return;
    }

    char key[128];
    unsigned long value;
    char unit[32];
    unsigned long memTotal = 0;
    unsigned long memAvailable = 0;
```

```
    while (fscanf(meminfo, "%127s %lu %31s", key, &value, unit) == 3) {
        if (strcmp(key, "MemTotal:") == 0) memTotal = value;
        else if (strcmp(key, "MemAvailable:") == 0) memAvailable = value;
        if (memTotal && memAvailable) break;
    }

    unsigned long memUsed = memTotal - memAvailable;

    printf("========== 内存使用情况 ==========\n");
    printf("内存总量: %lu kB\n", memTotal);
    printf("已使用内存: %lu kB\n", memUsed);
    printf("可用内存: %lu kB\n\n", memAvailable);

    double load1, load5, load15;
    if (fscanf(loadavg, "%lf %lf %lf", &load1, &load5, &load15) == 3) {
        printf("========== 平均负载 ==========\n");
        printf("1 分钟平均负载 : %.2f\n", load1);
        printf("5 分钟平均负载 : %.2f\n", load5);
        printf("15 分钟平均负载: %.2f\n", load15);
    } else {
        printf("读取平均负载失败。\n");
    }

    fclose(meminfo);
    fclose(loadavg);
}

int main(int argc, char *argv[]) {
    if (argc == 1) {
        func_A();
        return 0;
    }
    if (argc >= 2) {
```

```
        if (strcmp(argv[1], "-s") == 0) {
            func_B();
            return 0;
        } else if (strcmp(argv[1], "-l") == 0) {
            func_C();
            return 0;
        }
    }
    printf("用法:\n");
    printf("./test1          查看 CPU 类型及型号、Linux 内核版本号\n");
    printf("./test1 -s       查看系统运行时间、CPU 三状态时间、进程数、上下文切换次数\n");
    printf("./test1 -l       查看内存使用情况、平均负载\n");
    return 0;
}
```

1.2.3 运行结果及解释

```
vboxuser@Ubuntu:~/OSLab/exp1$ ./test1
========== CPU 类型及型号 ==========
AMD Ryzen 9 8945HX with Radeon Graphics

========== Linux 内核版本号 ==========
Linux version 6.17.0-19-generic (buildd@lcy02-amd64-019) (x86_64-linux-gnu-gcc-13 (Ubuntu 13.3.0-6ubuntu2~24.04.1) 13.3.0, GNU ld (GNU Binutils for Ubuntu) 2.42) #19~24.04.2-Ubuntu SMP PREEMPT_DYNAMIC Fri Mar  6 23:08:46 UTC 2
```

程序输出了 CPU 类型及型号和 Linux 内核版本号。

实验结果中显示，当前虚拟机系统的 CPU 型号为：AMD Ryzen 9 8945HX with Radeon Graphics

同时还输出了 Linux 内核版本信息。通过与 cat /proc/version 命令结果进行对比，可以确认程序成功读取并正确输出了 /proc/version 文件内容。

```
vboxuser@Ubuntu:~/OSLab/exp1$ ./test1 -s
========== 系统最近一次启动以来经历的时间 ==========
0时3分8秒

========== CPU 三种状态时间花费 ==========
用户态时间: 1517
内核态时间: 4579
空闲态时间: 22898

========== 系统启动以来的进程与上下文切换信息 ==========
系统创建的进程数: 3125
上下文转换次数: 279550
```

程序输出了系统最近一次启动以来经历的时间，以及 CPU 在用户态、内核态、空闲态三种状态下花费的时间，同时输出了系统启动以来创建的进程数和上下文切换次数。这些数据表明系统在运行过程中，大部分时间处于空闲状态，当前虚拟机负载较低。

```
vboxuser@Ubuntu:~/OSLab/exp1$ ./test1 -l
========== 内存使用情况 ==========
内存总量: 4008740 kB
已使用内存: 1082796 kB
可用内存: 2925944 kB

========== 平均负载 ==========
1分钟平均负载 : 0.88
5分钟平均负载 : 0.95
15分钟平均负载: 0.42
```

程序输出了当前系统的内存使用情况以及平均负载。

从结果可以看出，当前系统可用内存较充足，系统平均负载较低，说明虚拟机运行状态较平稳。

1.3 实验总结

通过本次实验，我初步掌握了 Linux /proc 虚拟文件系统的基本结构及其作用，了解了 Linux 系统中许多状态信息都是通过 /proc 文件系统动态提供的。

在实验过程中，我学会了使用 C 语言通过文件操作函数读取 /proc 目录下的相关文件，并对所需字段进行提取和格式化输出。

通过对 /proc/cpuinfo、/proc/version、/proc/uptime、/proc/stat、/proc/meminfo 和 /proc/loadavg 等文件的读取，我进一步理解了 CPU 信息、系统运行时间、进程统计信息、内存使用情况和系统负载等内容的来源。

本实验加深了我对 Linux 操作系统运行机制的认识，也为后续操作系统实验打下了基础。

实验 2　软中断通信实验

2.1 实验目的

掌握软、硬中断的基本原理;
掌握中断信号的使用；
学会进程的创建方法；
使用软中断信号来实现进程间的通信。

2.2 实验内容

父进程创建两个子进程，等待键盘终端输入“Ctrl+\”组合键，向父进程发送 SIGQUIT 硬中断信号；
父进程接收硬中断信号后，向其两个子进程分别发送整数值为 16 和 17 软中断信号；
子进程获得对应软中断信号后，终止运行；
父进程等待两个子进程终止后，然后自我终止。

2.2.1 设计思想及算法流程

本实验采用父子进程结构实现软中断通信。
程序首先通过 fork()创建第一个子进程。第一个子进程通过 signal(16,stop1)注册 16 号信号的处理函数，并进入等待信号状态。
父进程继续通过 fork()创建第二个子进程。第二个子进程通过 signal(17,stop2)注册 17 号信号的处理函数，并进入等待信号状态。
父进程通过 signal(SIGQUIT,quit)注册 SIGQUIT 信号的处理函数。当用户在终端按下 Ctrl+\后，父进程接收到 SIGQUIT 信号，并执行 quit()函数。
在 quit()函数中，父进程通过 kill(c1,16)向第一个子进程发送 16 号软中断信号，通过 kill(c2,17)向第二个子进程发送 17 号软中断信号。两个子进程收到对应信号后分别执行 stop1()和 stop2()函数，输出结束信息并调用 exit(0)退出。
最后，父进程通过两次 wait()等待两个子进程全部结束，然后输出父进程结束信息并退出。
程序总体流程如下：
（1）父进程创建第一个子进程。
（2）第一个子进程注册 16 号信号处理函数，并等待信号。
（3）父进程创建第二个子进程。
（4）第二个子进程注册 17 号信号处理函数，并等待信号。

（5）父进程注册 SIGQUIT 信号处理函数，并等待 Ctrl+\。

（6）用户按下 Ctrl+\，父进程收到 SIGQUIT 信号。

（7）父进程向第一个子进程发送 16 号信号。

（8）父进程向第二个子进程发送 17 号信号。

（9）两个子进程收到信号后分别退出。

（10）父进程等待两个子进程结束后退出。

2.2.2 源程序清单

```
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <signal.h>
#include <sys/wait.h>

pid_t c1, c2;

void stop1(int sig)
{
    printf("child process 1 is killed by signal %d\n", sig);
    exit(0);
}
void stop2(int sig)
{
    printf("child process 2 is killed by signal %d\n", sig);
    exit(0);
}
void quit(int sig)
{
    int st;
    printf("parent process receives SIGQUIT\n");
    kill(c1, 16);
    kill(c2, 17);
    wait(&st);
    wait(&st);
```

```
    printf("parent process is killed\n");
    exit(0);
}
int main()
{
    c1 = fork();
    if (c1 < 0)
    {
        perror("fork");
        exit(1);
    }
    if (c1 == 0)
    {
        signal(SIGQUIT, SIG_IGN);
        signal(16, stop1);
        printf("child process 1 is running\n");
        while (1)
        {
            pause();
        }
    }
    c2 = fork();

    if (c2 < 0)
    {
        perror("fork");
        exit(1);
    }
    if (c2 == 0)
    {
        signal(SIGQUIT, SIG_IGN);
        signal(17, stop2);
        printf("child process 2 is running\n");
        while (1)
```

```
        {
            pause();
        }
    }
    signal(SIGQUIT, quit);
    printf("parent process is waiting for SIGQUIT\n");
    while (1)
    {
        pause();
    }
    return 0;
}
```

2.2.3 运行结果及解释

```
vboxuser@Ubuntu:~/OSLab/exp2$ ./test2
child process 1 is running
parent process is waiting for SIGQUIT
child process 2 is running
```

程序输出了两个子进程开始运行的信息，同时父进程进入等待 SIGQUIT 信号的状态。

```
vboxuser@Ubuntu:~/OSLab/exp2$ ./test2
child process 1 is running
parent process is waiting for SIGQUIT
child process 2 is running
^\parent process receives SIGQUIT
child process 1 is killed by signal 16
child process 2 is killed by signal 17
parent process is killed
```

按下 Ctrl+\ 后，父进程成功接收到 SIGQUIT 信号，并输出 parent process receives SIGQUIT。

随后，父进程分别向两个子进程发送 16 号和 17 号软中断信号。第一个子进程收到 16 号信号后，输出 child process 1 is killed by signal 16，并调用 exit(0) 退出。第二个子进程收到 17 号信号后，输出 child process 2 is killed by signal 17，并调用 exit(0) 退出。

两个子进程结束后，父进程通过 wait() 等待子进程终止，最后输出 parent process is killed 并退出。

实验结果表明，父进程能够正确接收键盘产生的 SIGQUIT 信号，并通过 kill() 向两个子进程发送不同的软中断信号。两个子进程能够根据收到的信号执行对应的处理函数并正常终止，程序运行结果符合实验要求。

2.3 实验总结

通过本次实验，我掌握了 Linux 系统中使用信号实现进程间通信的基本方法，进一步理解了软中断、硬中断以及信号处理机制的作用。

在实验过程中，我使用 fork() 创建了两个子进程，使用 signal() 为父进程和子进程分别设置对应的信号处理函数。父进程接收到键盘输入 Ctrl+\ 产生的 SIGQUIT 信号后，通过 kill() 向两个子进程分别发送 16 号和 17 号软中断信号。两个子进程收到对应信号后执行处理函数并退出，父进程再通过 wait() 等待两个子进程结束后退出。

本实验加深了我对进程创建、信号发送、信号处理和进程等待过程的理解。通过运行结果可以看出，信号机制可以实现父进程对子进程的控制，也可以用于进程之间的简单通信，为后续学习进程同步与进程通信相关内容打下了基础。

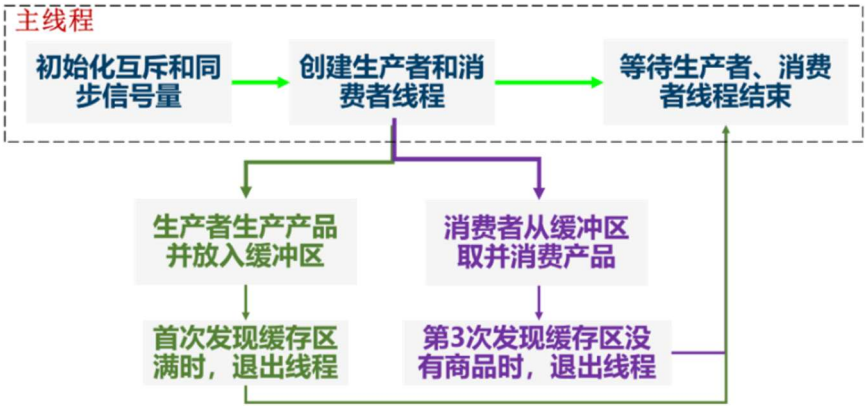
实验 3　线程同步实验

3.1 实验目的

掌握操作系统并发的概念
掌握 Linux 多线程编程机制
理解多线程同步和互斥
掌握信号量

3.2 实验内容

编写 Linux 多线程程序，通过信号量实现生产者—消费者问题。



3.2.1 设计思想及算法流程

本实验采用一个生产者线程、一个消费者线程和一个有限缓冲区实现生产者—消费者问题。

程序中定义大小为 5 的缓冲区 buf，用 in 表示生产者放入产品的位置，用 out 表示消费者取出产品的位置，用 cnt 记录当前缓冲区中的产品数量。

程序使用三个信号量实现线程同步与互斥：

mutex：互斥信号量，初始值为 1，用于保证同一时刻只有一个线程访问缓冲区。

empty：空缓冲区数量信号量，初始值为缓冲区大小 SIZE，表示当前可以放入产品的位置数量。

full：产品数量信号量，初始值为 0，表示当前缓冲区中已有产品的数量。

生产者线程运行时，先判断缓冲区是否还有空位。如果缓冲区已满，则输出提示信息并退出线程。如果缓冲区未满，则进入临界区，将产品放入缓冲区，更新 in 和 cnt 的值，然后释放互斥信号量，并增加 full 信号量。

消费者线程运行时，先判断缓冲区中是否存在产品。如果缓冲区为空，则记录发现空

缓冲区的次数。当消费者第三次发现缓冲区为空时，输出提示信息并退出线程。如果缓冲区中存在产品，则进入临界区，从缓冲区中取出产品，更新 out 和 cnt 的值，然后释放互斥信号量，并增加 empty 信号量。

主线程负责初始化三个信号量，创建生产者线程和消费者线程，并通过 pthread_join() 等待两个线程结束。两个线程都结束后，主线程销毁信号量并退出。

程序总体流程如下：

（1）主线程声明并初始化信号量 mutex、empty、full。

（2）主线程创建生产者线程和消费者线程。

（3）生产者线程不断生产产品，并尝试放入缓冲区。

（4）消费者线程不断从缓冲区取出产品并消费。

（5）生产者放入产品前需要判断缓冲区是否已满。

（6）消费者取出产品前需要判断缓冲区是否为空。

（7）生产者首次发现缓冲区满时退出线程。

（8）消费者第三次发现缓冲区为空时退出线程。

（9）主线程等待两个子线程结束。

（10）主线程释放信号量资源并退出。

3.2.2 源程序清单

```
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <pthread.h>
#include <semaphore.h>
#include <errno.h>

#define SIZE 5

int buf[SIZE];
int in = 0;
int out = 0;
int cnt = 0;

sem_t mutex;
sem_t empty;
```

```
sem_t full;

void show_buf()
{
	int i;
	printf("buffer: ");
	for (i = 0; i < SIZE; i++)
	{
		printf("%2d ", buf[i]);
	}
	printf("\n\n");
}

void *prod(void *arg)
{
	int item = 1;

	while (1)
	{
		sleep(1);

		if (sem_trywait(&empty) != 0)
		{
			if (errno == EAGAIN)
			{
				printf("buffer is full, producer exits\n");
				pthread_exit("producer finished");
			}
			else
			{
				perror("sem_trywait empty");
				pthread_exit("producer error");
			}
		}
```

```
            sem_wait(&mutex);

            buf[in] = item;
            in = (in + 1) % SIZE;
            cnt++;

            printf("producer produces item %d\n", item);
            printf("buffer count = %d\n", cnt);
            show_buf();

            item++;

            sem_post(&mutex);
            sem_post(&full);
        }
}

void *cons(void *arg)
{
        int item;
        int emp_cnt = 0;

        while (1)
        {
            sleep(2);

            if (sem_trywait(&full) != 0)
            {
                if (errno == EAGAIN)
                {
                    emp_cnt++;
                    printf("buffer is empty, consumer finds empty %d time(s)\n", emp_cnt);
```

```
                    if (emp_cnt >= 3)
                    {
                        printf("consumer finds empty 3 times, consumer exits\n");
                        pthread_exit("consumer finished");
                    }

                    continue;
                }
                else
                {
                    perror("sem_trywait full");
                    pthread_exit("consumer error");
                }
            }

            sem_wait(&mutex);

            item = buf[out];
            buf[out] = 0;
            out = (out + 1) % SIZE;
            cnt--;

            printf("consumer consumes item %d\n", item);
            printf("buffer count = %d\n", cnt);
            show_buf();

            sem_post(&mutex);
            sem_post(&empty);
        }
    }

    int main()
    {
        pthread_t t1, t2;
```

```
void *ret1;
void *ret2;
int res;

sem_init(&mutex, 0, 1);
sem_init(&empty, 0, SIZE);
sem_init(&full, 0, 0);

printf("producer-consumer test begins\n\n");

res = pthread_create(&t1, NULL, prod, NULL);
if (res != 0)
{
    perror("producer thread create failed");
    exit(1);
}

res = pthread_create(&t2, NULL, cons, NULL);
if (res != 0)
{
    perror("consumer thread create failed");
    exit(1);
}

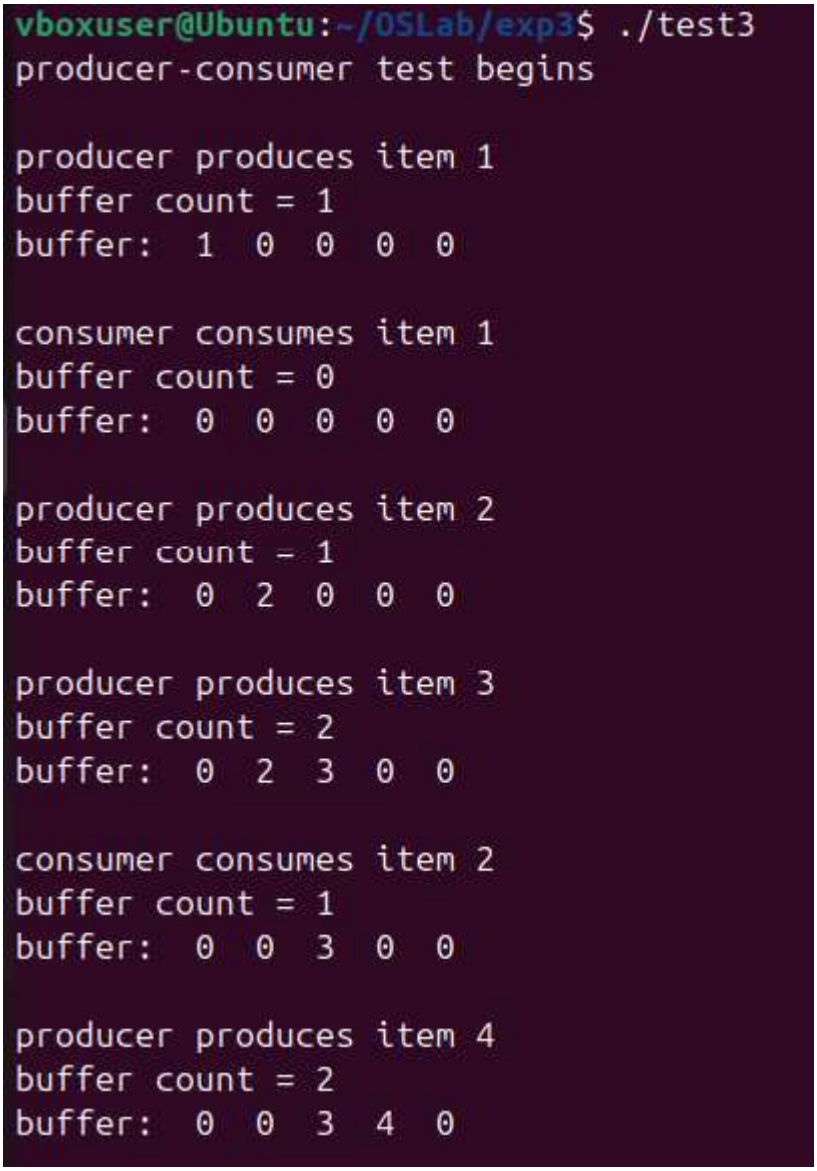
pthread_join(t1, &ret1);
pthread_join(t2, &ret2);

printf("producer thread returns: %s\n", (char *)ret1);
printf("consumer thread returns: %s\n", (char *)ret2);
printf("main thread exits\n");

sem_destroy(&mutex);
sem_destroy(&empty);
sem_destroy(&full);
```

```
    return 0;
}
```

3.2.3 运行结果及解释



```
producer produces item 5
buffer count = 3
buffer:  0  0  3  4  5

consumer consumes item 3
buffer count = 2
buffer:  0  0  0  4  5

producer produces item 6
buffer count = 3
buffer:  6  0  0  4  5

producer produces item 7
buffer count = 4
buffer:  6  7  0  4  5

producer produces item 8
buffer count = 5
buffer:  6  7  8  4  5

consumer consumes item 4
buffer count = 4
buffer:  6  7  8  0  5

producer produces item 9
buffer count = 5
buffer:  6  7  8  9  5
```

```
consumer consumes item 5
buffer count = 4
buffer:  6  7  8  9  0

producer produces item 10
buffer count = 5
buffer:  6  7  8  9 10

buffer is full, producer exits
consumer consumes item 6
buffer count = 4
buffer:  0  7  8  9 10

consumer consumes item 7
buffer count = 3
buffer:  0  0  8  9 10

consumer consumes item 8
buffer count = 2
buffer:  0  0  0  9 10

consumer consumes item 9
buffer count = 1
buffer:  0  0  0  0 10

consumer consumes item 10
buffer count = 0
buffer:  0  0  0  0  0
```

从运行结果可以看出，生产者线程不断生产产品，并将产品放入缓冲区；消费者线程从缓冲区中取出产品并进行消费。每次生产或消费后，程序都会输出当前缓冲区中的产品数量以及缓冲区内容。

当生产者生产速度快于消费者消费速度时，缓冲区中的产品数量逐渐增加。当缓冲区被放满后，生产者线程输出 buffer is full, producer exits

说明生产者检测到缓冲区已满，因此停止生产并退出线程。

```
buffer is empty, consumer finds empty 1 time(s)
buffer is empty, consumer finds empty 2 time(s)
buffer is empty, consumer finds empty 3 time(s)
consumer finds empty 3 times, consumer exits
producer thread returns: producer finished
consumer thread returns: consumer finished
main thread exits
```

生产者线程退出后，消费者线程继续消费缓冲区中剩余的产品。当缓冲区中的产品被全部取出后，消费者线程开始检测到缓冲区为空。消费者第三次发现缓冲区为空时，输出 buffer is empty, consumer finds empty 3 time(s)

consumer finds empty 3 times, consumer exits

说明消费者线程已经按照实验要求退出。

最后，主线程等待生产者线程和消费者线程结束，并输出：

producer thread returns: producer finished

consumer thread returns: consumer finished

main thread exits

说明主线程成功回收了两个子线程的资源，并正常结束程序。

实验结果表明，程序能够正确创建生产者线程和消费者线程，并通过信号量实现对缓冲区的同步和互斥访问。生产者能够在缓冲区未满时放入产品，消费者能够在缓冲区非空时取出产品，程序运行结果符合实验要求。

3.3 实验总结

通过本次实验，我掌握了 Linux 多线程程序设计的基本方法，进一步理解了线程同步、线程互斥以及信号量机制的作用。

在实验过程中，我使用 pthread_create() 创建生产者线程和消费者线程，使用 pthread_join() 等待线程结束，并使用 sem_init()、sem_wait()、sem_post() 和 sem_destroy() 对信号量进行初始化、等待、释放和销毁。程序通过 mutex 信号量实现对缓冲区的互斥访问，通过 empty 和 full 信号量控制生产者和消费者之间的同步关系。

本实验加深了我对生产者—消费者问题的理解。通过运行结果可以看出，如果没有同步与互斥控制，多个线程同时访问共享缓冲区时可能会产生错误结果。使用信号量后，生产者和消费者可以按照正确顺序访问缓冲区，从而保证程序运行的正确性。本实验也为后续学习进程同步、线程通信和并发控制打下了基础。

实验 4　进程调度实验

4.1 实验目的

了解短程、中程、长程调度的区别
掌握短进程调度及其策略

4.2 实验内容

代码实现短进程调度算法
先来先服务（First Come First Serve）
轮转（Round Robin）
基于优先级的调度算法。

4.2.1 设计思想及算法流程

本实验采用结构体 P 描述进程信息，主要字段包括：
id：任务编号
arr：到达时间
need：需要执行时间
left：剩余执行时间
done：是否完成
p0：初始优先级
prio：当前优先级
程序总体设计思想如下：
（1）数据初始化
程序首先定义 5 个初始任务，并通过 reset_p() 将原始任务复制到当前调度数组中。为了保证程序对不同任务排列情况的适应性，在 reset_p() 中调用 sort_arr()，按照到达时间 arr 对任务进行排序。
同时，调用 init() 将输出矩阵初始化为空格，用于后续记录调度过程。
（2）调度过程记录

程序使用二维字符数组 mp[N][T] 记录调度情况：
行表示任务
列表示时间
若某任务在某时刻执行，则在对应位置标记 #

最终通过 show_map() 输出调度图，从而得到 task 纵向、time 横向的执行结果。

（3）FCFS 算法设计

在先来先服务调度中，程序在当前时刻从前往后扫描任务数组，选择第一个“已到达且未完成”的任务。

一旦选中，就让该任务连续执行直到完成，因此 FCFS 属于非抢占式调度。

在执行过程中，程序不断减少该任务的剩余执行时间 left，并在调度图中记录执行痕迹，直到其完成后再处理下一个任务。

（4）RR 算法设计

在时间片轮转调度中，程序使用数组模拟就绪队列，并用 head 和 tail 表示队头和队尾。

每次调度时，先将当前时刻新到达的任务加入队列，然后从队头取出一个任务执行一个时间片。

执行结束后：

若任务已完成，则将其标记为完成

若任务尚未完成，则将其重新加入队尾

这样就实现了“每次运行一个时间片，未完成则回到队尾”的轮转机制。

为了符合本实验的展示要求，程序只把当前时刻 arr == now 的新到达任务加入队列，从而保证新到达任务先入队，再将未完成的当前任务放回队尾。

（5）基于优先级的调度算法设计

在优先级调度中，程序在开始时为每个任务随机生成一个 1 到 5 的初始优先级，并存入 p0，同时令当前优先级 prio = p0。

在每个调度周期中，程序从所有“已到达、未完成、当前优先级大于 0”的任务中，选取当前优先级最高的任务运行 1 个时间片。

执行后：

该任务剩余执行时间 left 减 1

当前优先级 prio 减 1

当某个任务 prio = 0 时，该任务本轮不再参与调度。

当所有未完成任务的当前优先级都为 0 时，程序调用 reset_prio()，将这些未完成任务的当前优先级恢复为初始优先级 p0，然后继续调度，直到所有任务全部完成。

（6）主程序流程

主函数通过循环菜单读取用户输入，根据用户选择分别执行：

f：先来先服务调度

r：轮循调度

p：基于优先级调度

q：退出程序

程序在执行每种调度前都重新初始化任务数据，以保证不同算法在相同初始条件下运行，便于比较。

4.2.2 源程序清单

```
#include <stdio.h>
#include <stdlib.h>
#include <time.h>

#define N 5
#define T 200

typedef struct
{
    int id;
    int arr;
    int need;
    int left;
    int done;
    int p0;
    int prio;
} P;

P src[N] =
{
    {1, 0, 3, 3, 0, 0, 0},
    {2, 2, 6, 6, 0, 0, 0},
    {3, 4, 4, 4, 0, 0, 0},
    {4, 6, 5, 5, 0, 0, 0},
    {5, 8, 2, 2, 0, 0, 0}
};

void sort_arr(P p[])
{
```

```
    int i, j;
    P tmp;
    for (i = 0; i < N - 1; i++)
    {
        for (j = 0; j < N - 1 - i; j++)
        {
            if (p[j].arr > p[j + 1].arr)
            {
                tmp = p[j];
                p[j] = p[j + 1];
                p[j + 1] = tmp;
            }
        }
    }
}

void reset_p(P p[])
{
    int i;
    for (i = 0; i < N; i++)
    {
        p[i] = src[i];
    }
    sort_arr(p);
}

int all_done(P p[])
{
    int i;
    for (i = 0; i < N; i++)
    {
        if (!p[i].done)
        {
            return 0;
```

```
            }
        }
        return 1;
}

void init(char mp[][T])
{
        int i, j;
        for (i = 0; i < N; i++)
        {
                for (j = 0; j < T; j++)
                {
                        mp[i][j] = ' ';
                }
        }
}

void show_task(P p[])
{
        int i;
        printf("task id      start    excute\n");
        for (i = 0; i < N; i++)
        {
                printf("task %2d: %6d %7d\n", p[i].id, p[i].arr, p[i].need);
        }
}

void show_p0(P p[])
{
        int i;
        printf("任务优先级依次为：");
        for (i = 0; i < N; i++)
        {
                printf("%3d", p[i].p0);
```

```
    }
    printf("\n");
}

void show_map(char mp[][T], int endt)
{
    int i, t;
    printf("time      :");
    for (t = 0; t <= endt; t++)
    {
        printf(" %2d", t);
    }
    printf("\n");
    for (i = 0; i < N; i++)
    {
        printf("task %2d:", i + 1);
        for (t = 0; t <= endt; t++)
        {
            printf(" %2c", mp[i][t]);
        }
        printf("\n");
    }
    printf("\n");
}

void run_f()
{
    P p[N];
    char mp[N][T];
    int now = 0;
    int pick, i;
    reset_p(p);
    init(mp);
    while (!all_done(p))
```

```
    {
        pick = -1;
        for (i = 0; i < N; i++)
        {
            if (!p[i].done && p[i].arr <= now)
            {
                pick = i;
                break;
            }
        }
        if (pick == -1)
        {
            now++;
            continue;
        }
        while (p[pick].left > 0)
        {
            mp[pick][now] = '#';
            p[pick].left--;
            now++;
        }
        p[pick].done = 1;
    }
    show_task(p);
    show_map(mp, now - 1);
}

void add_now(P p[], int q[], int* tail, int inq[], int now)
{
    int i;
    for (i = 0; i < N; i++)
    {
        if (!p[i].done && !inq[i] && p[i].left > 0 && p[i].arr == now)
        {
```

```
                q[*tail] = i;
                (*tail)++;
                inq[i] = 1;
            }
        }
}

void run_r()
{
    P p[N];
    char mp[N][T];
    int q[T];
    int inq[N] = { 0 };
    int head = 0, tail = 0;
    int now = 0;
    int idx;
    reset_p(p);
    init(mp);
    add_now(p, q, &tail, inq, 0);
    while (!all_done(p))
    {
        if (head == tail)
        {
            now++;
            add_now(p, q, &tail, inq, now);
            continue;
        }
        idx = q[head++];
        inq[idx] = 0;
        if (p[idx].done || p[idx].left <= 0)
        {
            continue;
        }
        mp[idx][now] = '#';
```

```
            p[idx].left--;
            now++;
            add_now(p, q, &tail, inq, now);
            if (p[idx].left == 0)
            {
                p[idx].done = 1;
            }
            else
            {
                q[tail++] = idx;
                inq[idx] = 1;
            }
        }
        show_task(p);
        show_map(mp, now - 1);
}

void reset_prio(P p[])
{
        int i;
        for (i = 0; i < N; i++)
        {
            if (!p[i].done)
            {
                p[i].prio = p[i].p0;
            }
        }
}

int zero(P p[], int now)
{
        int i, has = 0;
        for (i = 0; i < N; i++)
        {
```

```
            if (!p[i].done && p[i].arr <= now)
            {
                has = 1;
                if (p[i].prio > 0)
                {
                    return 0;
                }
            }
        }
        return has;
}

void run_p()
{
        P p[N];
        char mp[N][T];
        int now = 0;
        int i, pick, best;

        reset_p(p);
        init(mp);

        srand((unsigned)time(NULL));
        for (i = 0; i < N; i++)
        {
            p[i].p0 = rand() % 5 + 1;
            p[i].prio = p[i].p0;
        }

        while (!all_done(p))
        {
            if (zero(p, now))
            {
                reset_prio(p);
```

```
        }
        pick = -1;
        best = -1;
        for (i = 0; i < N; i++)
        {
            if (!p[i].done && p[i].arr <= now && p[i].prio > 0)
            {
                if (p[i].prio >= best)
                {
                    best = p[i].prio;
                    pick = i;
                }
            }
        }
        if (pick == -1)
        {
            now++;
            continue;
        }
        mp[pick][now] = '#';
        p[pick].left--;
        p[pick].prio--;
        now++;

        if (p[pick].left == 0)
        {
            p[pick].done = 1;
        }
    }

    show_task(p);
    show_p0(p);
    show_map(mp, now - 1);
}
```

```
int main()
{
    char c;
    while (1)
    {
        printf("please choice the schedule measure:\n");
        printf("p :  基于优先级的调度策略\n");
        printf("f :  先到先服务的调度策略\n");
        printf("r :  轮循的调度策略\n");
        printf("q :  退出\n");
        printf("choice = ");
        c = getchar();
        if (c == '\n')
        {
            c = getchar();
        }
        while (getchar() != '\n');

        switch (c)
        {
        case 'f':
            run_f();
            break;
        case 'r':
            run_r();
            break;
        case 'p':
            run_p();
            break;
        case 'q':
            return 0;
        default:
            printf("please input the true symbol(p or f or r)!\n\n");
```

```
                break;
            }
        }
        return 0;
}
```

4.2.3 运行结果及解释

```
please choice the schedule measure:
p : 基于优先级的调度策略
f : 先到先服务的调度策略
r : 轮循的调度策略
q : 退出
choice = f
task id   start  excute
task  1:      0       3
task  2:      2       6
task  3:      4       4
task  4:      6       5
task  5:      8       2
time   :  0  1  2  3  4  5  6  7  8  9 10 11 12 13 14 15 16 17 18 19
task  1:  #  #  #
task  2:           #  #  #  #  #  #
task  3:                          #  #  #  #
task  4:                                   #  #  #  #  #
task  5:                                                  #  #
```

运行./test4 后选择 f，程序输出了 5 个任务的基本信息，包括任务编号、到达时间和需要执行时间。

在 FCFS 调度中，各任务按照到达时间先后顺序依次执行，并且一旦获得处理机，就会连续运行直到结束，因此属于非抢占式调度。

从调度图可以明显看出，各任务的执行时间段是连续的，没有被打断，符合先来先服务调度算法先到先执行、执行后不被抢占的特点。

```
please choice the schedule measure:
p : 基于优先级的调度策略
f : 先到先服务的调度策略
r : 轮循的调度策略
q : 退出
choice = f
task id   start  excute
task  1:      0       3
task  2:      2       6
task  3:      4       4
task  4:      6       5
task  5:      8       2
time   :  0  1  2  3  4  5  6  7  8  9 10 11 12 13 14 15 16 17 18 19
task  1:  #  #  #
task  2:           #  #  #  #  #  #
task  3:                           #  #  #  #
task  4:                                    #  #  #  #  #
task  5:                                                   #  #
```

运行./test4 后选择 r，程序采用时间片轮转调度算法进行处理。

在该算法中，系统每次只为一个任务分配一个时间片，时间片结束后若任务尚未完成，则将其重新放回队尾，等待下一轮调度，因此属于抢占式调度。

从结果可以看出，各任务的执行过程呈现出明显的交替特征，不再像 FCFS 那样连续执行，而是分散在多个时间点上。这说明程序已经正确实现了时间片轮转调度的基本思想。同时，由于程序采用“新到达任务先入队，当前未完成任务再回到队尾”的方式，因此在新任务到达时，任务执行顺序也会发生相应变化。

```
please choice the schedule measure:
p : 基于优先级的调度策略
f : 先到先服务的调度策略
r : 轮循的调度策略
q : 退出
choice = p
task id   start  excute
task  1:      0       3
task  2:      2       6
task  3:      4       4
task  4:      6       5
task  5:      8       2
任务优先级依次为：  5  1  4  5  3
time   :  0  1  2  3  4  5  6  7  8  9 10 11 12 13 14 15 16 17 18 19
task  1:  #  #  #
task  2:           #                                 #  #  #  #  #
task  3:              #  #                    #     #
task  4:                    #  #     #     #     #
task  5:                          #     #
```

```
please choice the schedule measure:
p : 基于优先级的调度策略
f : 先到先服务的调度策略
r : 轮循的调度策略
q : 退出
choice = p
task id   start  excute
task  1:      0       3
task  2:      2       6
task  3:      4       4
task  4:      6       5
task  5:      8       2
任务优先级依次为：  4  3  5  1  2
time    :  0  1  2  3  4  5  6  7  8  9 10 11 12 13 14 15 16 17 18 19
task  1:  #  #                        #
task  2:        #  #                          #  #  #     #
task  3:              #  #  #  #
task  4:                                    #           #     #  #  #
task  5:                          #     #
```

运行./test4 后选择 p，程序会先随机生成并输出各任务的初始优先级，然后按照优先级调度策略运行。

在该算法中，每个任务都有一个当前优先级 prio，每执行一个调度周期后，当前运行任务的优先级减 1；当某任务优先级减为 0 时，在本轮中不再参与调度；当所有未完成任务当前优先级都为 0 时，再恢复为各自的初始优先级。

本次实验中进行了两次优先级调度测试。两次实验结果都表明，基于优先级的调度不仅受到任务到达时间的影响，还受到优先级变化的影响。与 FCFS 和 RR 相比，优先级调度的执行分布更加复杂，不同任务的执行顺序会随着当前优先级的变化而动态调整。这说明程序已经较好地实现了优先级递减、优先级归零后恢复的调度机制。

4.3 实验总结

通过本次实验，我完成了先来先服务调度算法、时间片轮转调度算法和基于优先级的调度算法的实现，并通过程序输出直观观察了三种调度方式的运行过程。

在实验中，我进一步理解了非抢占式调度和抢占式调度的区别。其中，FCFS 的执行过程最简单，任务按到达顺序依次运行且不会被打断；RR 通过时间片机制实现了任务之间的轮流执行；优先级调度则在任务到达时间基础上进一步引入优先级因素，使调度过程更具动态性。

通过本次实验，我不仅掌握了三种常见进程调度算法的基本原理，也提高了使用 C 语言实现调度模型和分析运行结果的能力，为后续操作系统相关实验打下了基础。

实验 5　内存管理实验

5.1 实验目的

了解虚拟内存与虚拟分页
理解页面置换的动机；
掌握并实现部分页面替换策略;
• 最优置换（OPT）
• 先进先出（FIFO）
• 最近最少使用（LRU）

5.2 实验内容

在 Linux 下编程实现虚存页面替换算法的模拟程序。
1.输出用户进程页数，分配的页框数；
2.根据进程页数，随机生成进程页访问序列；
3.选择 OPT、FIFO、LRU 替换算法；
4.输出相应替换算法页面置换流程。

5.2.1 设计思想及算法流程

1. 设计思想
本实验通过 C 语言模拟虚拟内存中的页面置换过程。程序输入进程页数和物理内存页框数，其中页框数必须小于进程页数。程序根据进程页数随机生成页面访问序列，并分别使用 OPT、FIFO、LRU 三种页面置换算法进行模拟。
程序使用数组保存页面访问序列，使用数组模拟物理内存中的页框状态。每次访问一个页面时，程序判断该页面是否已经在页框中。如果页面已在页框中，则本次访问为命中；如果页面不在页框中，则发生缺页，并根据当前选择的页面置换算法决定是否装入空页框或替换已有页面。
OPT 算法根据未来访问情况选择最晚被访问或不再被访问的页面进行置换；FIFO 算法按照页面进入内存的先后顺序进行置换；LRU 算法根据页面最近一次被访问的时间进行置换。程序在每一步输出当前访问页、命中或缺页状态以及页框内容，最后统计并输出缺页次数。
2. 算法流程
开始
　　输入进程页数 pageCount

输入物理内存页框数 frameCount

判断 frameCount 是否小于 pageCount

若不满足条件，则输出错误信息并结束程序

根据 pageCount 生成页面访问序列

访问序列长度 = pageCount * MULTIPLE

每个访问页号在 0 到 pageCount - 1 之间随机生成

输出进程页数、页框数和页面访问序列

循环显示算法选择菜单

输入 o，执行 OPT 页面置换算法

输入 f，执行 FIFO 页面置换算法

输入 r，执行 LRU 页面置换算法

输入 q，退出程序

执行页面置换算法时：

初始化页框为空

从前到后依次读取页面访问序列

判断当前访问页是否已在页框中

若在页框中，则输出 Hit 和当前页框状态

若不在页框中，则缺页次数加 1

若存在空页框，则将页面装入空页框

若不存在空页框，则根据对应算法选择页面进行置换

输出 Fault 和当前页框状态

输出该算法的缺页次数

结束

3. 三种页面置换算法说明

OPT 算法：访问页面发生缺页且页框已满时，查找当前页框中每个页面在未来页面访问序列中的下一次出现位置，选择下一次使用时间最晚的页面进行置换。如果某个页面之后不再被访问，则优先置换该页面。

FIFO 算法：访问页面发生缺页且页框已满时，置换最早进入内存的页面。程序使用循环指针记录当前应被替换的页框位置，每发生一次页面置换后，指针移动到下一个页框。

LRU 算法：访问页面发生缺页且页框已满时，置换最近最少使用的页面。程序使用 lastUse 数组记录每个页框中页面最近一次被访问的时间，选择访问时间最早的页面进行置换。

5.2.2 源程序清单

test5.c

```
#include <stdio.h>
#include <stdlib.h>
#include <time.h>

#define MULTIPLE 4

void printFrames(int frames[], int frameCount)
{
	int i;
	printf("[");
	for (i = 0; i < frameCount; i++) {
		if (frames[i] == -1)
			printf(" - ");
		else
			printf(" %d ", frames[i]);
	}
	printf("]");
}

int isInFrames(int frames[], int frameCount, int page)
{
	int i;
	for (i = 0; i < frameCount; i++) {
		if (frames[i] == page)
			return i;
	}
	return -1;
}

void initFrames(int frames[], int frameCount)
{
```

```
    int i;
    for (i = 0; i < frameCount; i++)
        frames[i] = -1;
}

void printReference(int ref[], int refCount)
{
    int i;
    printf("Page reference sequence:\n");
    for (i = 0; i < refCount; i++)
        printf("%d ", ref[i]);
    printf("\n\n");
}

void opt(int pageCount, int frameCount, int ref[], int refCount)
{
    int frames[100];
    int faults = 0;
    int i, j;

    initFrames(frames, frameCount);

    printf("\nOPT replacement process:\n");
    printf("Step\tPage\tResult\tFrames\n");

    for (i = 0; i < refCount; i++) {
        int page = ref[i];
        int hit = isInFrames(frames, frameCount, page);

        if (hit != -1) {
            printf("%d\t%d\tHit\t", i + 1, page);
            printFrames(frames, frameCount);
            printf("\n");
            continue;
```

```
}

faults++;

int empty = isInFrames(frames, frameCount, -1);
if (empty != -1) {
    frames[empty] = page;
} else {
    int replaceIndex = -1;
    int farthest = -1;

    for (j = 0; j < frameCount; j++) {
        int k;
        int nextUse = refCount + 1;

        for (k = i + 1; k < refCount; k++) {
            if (ref[k] == frames[j]) {
                nextUse = k;
                break;
            }
        }

        if (nextUse > farthest) {
            farthest = nextUse;
            replaceIndex = j;
        }
    }

    frames[replaceIndex] = page;
}

printf("%d\t%d\tFault\t", i + 1, page);
printFrames(frames, frameCount);
printf("\n");
```

```
    }

    printf("OPT page faults: %d\n", faults);
}

void fifo(int pageCount, int frameCount, int ref[], int refCount)
{
    int frames[100];
    int faults = 0;
    int pointer = 0;
    int i;

    initFrames(frames, frameCount);

    printf("\nFIFO replacement process:\n");
    printf("Step\tPage\tResult\tFrames\n");

    for (i = 0; i < refCount; i++) {
        int page = ref[i];
        int hit = isInFrames(frames, frameCount, page);

        if (hit != -1) {
            printf("%d\t%d\tHit\t", i + 1, page);
            printFrames(frames, frameCount);
            printf("\n");
            continue;
        }

        faults++;
        frames[pointer] = page;
        pointer = (pointer + 1) % frameCount;

        printf("%d\t%d\tFault\t", i + 1, page);
        printFrames(frames, frameCount);
```

```
            printf("\n");
        }

        printf("FIFO page faults: %d\n", faults);
}

void lru(int pageCount, int frameCount, int ref[], int refCount)
{
        int frames[100];
        int lastUse[100];
        int faults = 0;
        int i, j;

        initFrames(frames, frameCount);
        for (i = 0; i < frameCount; i++)
            lastUse[i] = -1;

        printf("\nLRU replacement process:\n");
        printf("Step\tPage\tResult\tFrames\n");

        for (i = 0; i < refCount; i++) {
            int page = ref[i];
            int hit = isInFrames(frames, frameCount, page);

            if (hit != -1) {
                lastUse[hit] = i;
                printf("%d\t%d\tHit\t", i + 1, page);
                printFrames(frames, frameCount);
                printf("\n");
                continue;
            }

            faults++;
```

```
            int empty = isInFrames(frames, frameCount, -1);
            if (empty != -1) {
                frames[empty] = page;
                lastUse[empty] = i;
            } else {
                int replaceIndex = 0;
                for (j = 1; j < frameCount; j++) {
                    if (lastUse[j] < lastUse[replaceIndex])
                        replaceIndex = j;
                }
                frames[replaceIndex] = page;
                lastUse[replaceIndex] = i;
            }

            printf("%d\t%d\tFault\t", i + 1, page);
            printFrames(frames, frameCount);
            printf("\n");
        }

        printf("LRU page faults: %d\n", faults);
}

int main()
{
        int pageCount, frameCount;
        int refCount;
        int ref[1000];
        int i;
        char choice;

        srand((unsigned int)time(NULL));

        printf("Enter number of pages in program: ");
        scanf("%d", &pageCount);
```

```
printf("Enter number of frames in physical memory: ");
scanf("%d", &frameCount);

if (frameCount >= pageCount) {
    printf("Error: number of frames must be less than number of pages.\n");
    return 0;
}

refCount = pageCount * MULTIPLE;

for (i = 0; i < refCount; i++)
    ref[i] = rand() % pageCount;

printf("\nNumber of pages in program: %d\n", pageCount);
printf("Number of frames in physical memory: %d\n", frameCount);
printReference(ref, refCount);

do {
    printf("\nPlease select an algorithm:\n");
    printf("o - OPT\n");
    printf("f - FIFO\n");
    printf("r - LRU\n");
    printf("q - Quit\n");
    printf("Your choice: ");
    scanf(" %c", &choice);

    switch (choice) {
        case 'o':
            opt(pageCount, frameCount, ref, refCount);
            break;
        case 'f':
            fifo(pageCount, frameCount, ref, refCount);
            break;
```

```
                case 'r':
                    lru(pageCount, frameCount, ref, refCount);
                    break;
                case 'q':
                    printf("Quit.\n");
                    break;
                default:
                    printf("Invalid choice.\n");
                    break;
            }
        } while (choice != 'q');

        return 0;
}
```

5.2.3 运行结果及解释

1. 编译与运行命令

cd ~/OSLab/exp5

gcc test5.c -o test5

./test5

2. 测试输入

Enter number of pages in program: 5

Enter number of frames in physical memory: 3

本次测试中，进程页数为 5，物理内存页框数为 3，满足页框数小于进程页数的实验要求。

程序随机生成的页面访问序列如下：

4 0 3 3 4 4 2 0 0 4 3 1 1 3 1 4 3 0 2 2

```
vboxuser@Ubuntu:~/OSLab/exp5$ gcc test5.c -o test5
./test5
Enter number of pages in program: 5
Enter number of frames in physical memory: 3

Number of pages in program: 5
Number of frames in physical memory: 3
Page reference sequence:
0 4 1 0 2 3 4 4 2 1 2 0 1 4 2 3 2 4 1 1
```

3. OPT 算法运行结果

选择 OPT 算法：

Your choice: o

运行结果中，OPT 算法缺页次数为：

OPT page faults: 8

OPT 算法根据未来页面访问情况选择置换页面。当发生缺页且页框已满时，程序选择未来最晚被访问或不再被访问的页面进行替换。因此，在本次页面访问序列下，OPT 算法的缺页次数为 8 次。

```
Please select an algorithm:
o - OPT
f - FIFO
r - LRU
q - Quit
Your choice: o

OPT replacement process:
Step    Page    Result  Frames
1       4       Fault   [ 4  -  - ]
2       0       Fault   [ 4  0  - ]
3       3       Fault   [ 4  0  3 ]
4       3       Hit     [ 4  0  3 ]
5       4       Hit     [ 4  0  3 ]
6       4       Hit     [ 4  0  3 ]
7       2       Fault   [ 4  0  2 ]
8       0       Hit     [ 4  0  2 ]
9       0       Hit     [ 4  0  2 ]
10      4       Hit     [ 4  0  2 ]
11      3       Fault   [ 4  0  3 ]
12      1       Fault   [ 4  1  3 ]
13      1       Hit     [ 4  1  3 ]
14      3       Hit     [ 4  1  3 ]
15      1       Hit     [ 4  1  3 ]
16      4       Hit     [ 4  1  3 ]
17      3       Hit     [ 4  1  3 ]
18      0       Fault   [ 0  1  3 ]
19      2       Fault   [ 2  1  3 ]
20      2       Hit     [ 2  1  3 ]
OPT page faults: 8
```

4. FIFO 算法运行结果

选择 FIFO 算法：

Your choice: f

运行结果中，FIFO 算法缺页次数为：

FIFO page faults: 9

FIFO 算法按照页面进入内存的先后顺序进行页面置换。当发生缺页且页框已满时，

程序置换最早进入页框的页面。因此，在本次页面访问序列下，FIFO 算法的缺页次数为 9 次。

```
Please select an algorithm:
o - OPT
f - FIFO
r - LRU
q - Quit
Your choice: f

FIFO replacement process:
Step    Page    Result  Frames
1       4       Fault   [ 4  -  - ]
2       0       Fault   [ 4  0  - ]
3       3       Fault   [ 4  0  3 ]
4       3       Hit     [ 4  0  3 ]
5       4       Hit     [ 4  0  3 ]
6       4       Hit     [ 4  0  3 ]
7       2       Fault   [ 2  0  3 ]
8       0       Hit     [ 2  0  3 ]
9       0       Hit     [ 2  0  3 ]
10      4       Fault   [ 2  4  3 ]
11      3       Hit     [ 2  4  3 ]
12      1       Fault   [ 2  4  1 ]
13      1       Hit     [ 2  4  1 ]
14      3       Fault   [ 3  4  1 ]
15      1       Hit     [ 3  4  1 ]
16      4       Hit     [ 3  4  1 ]
17      3       Hit     [ 3  4  1 ]
18      0       Fault   [ 3  0  1 ]
19      2       Fault   [ 3  0  2 ]
20      2       Hit     [ 3  0  2 ]
FIFO page faults: 9
```

5. LRU 算法运行结果

选择 LRU 算法：

Your choice: r

运行结果中，LRU 算法缺页次数为：

LRU page faults: 9

LRU 算法根据页面最近一次被访问的时间进行置换。当发生缺页且页框已满时，程序置换最长时间未被访问的页面。因此，在本次页面访问序列下，LRU 算法的缺页次数为 9 次。

```
Please select an algorithm:
o - OPT
f - FIFO
r - LRU
q - Quit
Your choice: r

LRU replacement process:
Step    Page    Result  Frames
1       4       Fault   [ 4  -  - ]
2       0       Fault   [ 4  0  - ]
3       3       Fault   [ 4  0  3 ]
4       3       Hit     [ 4  0  3 ]
5       4       Hit     [ 4  0  3 ]
6       4       Hit     [ 4  0  3 ]
7       2       Fault   [ 4  2  3 ]
8       0       Fault   [ 4  2  0 ]
9       0       Hit     [ 4  2  0 ]
10      4       Hit     [ 4  2  0 ]
11      3       Fault   [ 4  3  0 ]
12      1       Fault   [ 4  3  1 ]
13      1       Hit     [ 4  3  1 ]
14      3       Hit     [ 4  3  1 ]
15      1       Hit     [ 4  3  1 ]
16      4       Hit     [ 4  3  1 ]
17      3       Hit     [ 4  3  1 ]
18      0       Fault   [ 4  3  0 ]
19      2       Fault   [ 2  3  0 ]
20      2       Hit     [ 2  3  0 ]
LRU page faults: 9
```

6. 结果分析

本次测试结果如下：

进程页数：5

页框数：3

页面访问序列：

4 0 3 3 4 4 2 0 0 4 3 1 1 3 1 4 3 0 2 2

OPT 缺页次数：8

FIFO 缺页次数：9

LRU 缺页次数：9

实验结果表明，程序能够根据输入的进程页数和页框数随机生成页面访问序列，并能够按照用户选择分别执行 OPT、FIFO、LRU 三种页面置换算法。三种算法均输出了每一步的访问页号、命中或缺页状态以及当前页框内容。

在本次测试序列中，OPT 算法缺页次数最少。原因是 OPT 算法在置换时利用未来页面访问信息，选择最晚再被访问或不再被访问的页面进行替换，因此能够减少缺页次数。FIFO 算法按照页面进入内存的顺序进行置换，实现简单，但没有考虑页面后续访问情况。LRU 算法根据最近使用情况进行置换，能够反映一定的访问局部性，但本次随机序列下缺页次数与 FIFO 相同。

综上，程序运行结果符合实验要求，能够完成虚拟内存页面置换算法的模拟。

5.3 实验总结

本实验完成了虚拟内存页面置换算法的模拟程序设计与实现。程序能够根据输入的进程页数和页框数随机生成页面访问序列，并分别执行 OPT、FIFO、LRU 三种页面置换算法，输出每一步的页面访问情况、命中或缺页状态、页框变化过程以及最终缺页次数。

通过实验可知，页面置换算法的核心在于缺页发生后如何选择被替换页面。OPT 算法依据未来访问情况进行置换，在理论上能够获得较少的缺页次数；FIFO 算法按照页面进入内存的先后顺序进行置换，实现简单；LRU 算法依据页面最近一次被访问的时间进行置换，能够体现程序访问的局部性特点。

本次测试中，进程页数为 5，页框数为 3，随机生成的页面访问序列为：

4 0 3 3 4 4 2 0 0 4 3 1 1 3 1 4 3 0 2 2

三种算法的缺页次数分别为：

OPT：8 次

FIFO：9 次

LRU：9 次

实验结果表明，程序能够正确完成页面置换过程模拟，三种算法的执行流程和缺页次数统计均符合页面置换算法的基本原理。通过本实验，加深了对虚拟内存、虚拟分页和页面置换机制的理解，也掌握了使用程序模拟操作系统内存管理策略的方法。

实验 6　系统调用

6.1 实验目的

学习如何往 Linux 系统内核中添加一个系统函数，编译修改后的内核后，在用户空间编写程序调用新添加的内核函数。

6.2 实验内容

在内核的 sys.c 文件中创建一个新的系统调用 mysyscall()，指定调用号为 335；编写一个用户空间程序来测试 mysyscall(335, 学号), 打印出学号。

6.2.1 设计思想及算法流程

1. 设计思想

本实验通过修改 Linux 5.10.1 内核源码，实现一个新的系统调用 mysyscall()。Linux 系统调用需要在内核函数定义、函数声明以及系统调用表中完成对应配置，用户程序才能通过系统调用号访问新增的内核函数。

本实验将新增系统调用号设置为 335，对应的系统调用名称为 mysyscall，内核函数名为 sys_mysyscall。该系统调用接收一个 unsigned long long 类型参数 a，并将该参数作为返回值返回。用户测试程序将学号作为参数传入系统调用，若程序能够正确输出学号，则说明新增系统调用配置成功。

2. 关键参数说明

原内核版本：Linux Ubuntu 6.17.0-22-generic

新内核版本：Linux Ubuntu 5.10.1

系统调用号：335

系统调用名称：mysyscall

内核函数名：sys_mysyscall

参数名称：a

参数类型：unsigned long long

参数作用：接收用户程序传入的学号并返回

3. 算法流程

开始

查看当前系统内核版本

下载并解压 Linux 5.10.1 内核源码

进入 Linux 5.10.1 内核源码目录

修改 kernel/sys.c 文件

添加 mysyscall 系统调用函数

参数为 unsigned long long a

返回值为 a

修改 include/linux/syscalls.h 文件

添加 sys_mysyscall 函数声明

修改 arch/x86/entry/syscalls/syscall_64.tbl 文件

添加系统调用号 335

建立系统调用号、系统调用名称和内核函数之间的对应关系

清除历史编译文件

生成内核编译配置

修改 .config 配置文件

编译 Linux 5.10.1 内核

编译并安装内核模块

安装新内核

更新 GRUB 引导菜单

重启系统

在 GRUB 菜单中选择 Linux 5.10.1 内核

使用 uname -a 验证新内核版本

编写用户空间测试程序 test5.c

使用 syscall(335, 学号) 调用新增系统调用

编译并运行测试程序

查看输出结果

结束

4. 系统调用添加过程

本实验主要修改以下三个内核文件：

kernel/sys.c

include/linux/syscalls.h

arch/x86/entry/syscalls/syscall_64.tbl

在 kernel/sys.c 文件中添加系统调用函数：

```
SYSCALL_DEFINE1(mysyscall, unsigned long long, a)
{
```

```
    return a;
}
```

在 include/linux/syscalls.h 文件中添加函数声明：

```
asmlinkage long sys_mysyscall(unsigned long long a);
```

在 arch/x86/entry/syscalls/syscall_64.tbl 文件中添加系统调用表项：

```
335 common mysyscall sys_mysyscall
```

其中，335 表示系统调用号，common 表示通用 64 位系统调用，mysyscall 表示系统调用名称，sys_mysyscall 表示内核中对应的系统调用函数。

5. 内核编译与安装流程

进入内核源码目录：

```
cd /usr/src/linux-5.10.1
```

清除历史编译数据：

```
sudo make mrproper
sudo make clean
```

复制当前内核配置并生成编译配置：

```
sudo cp /boot/config-$(uname -r) .config
sudo make oldconfig
```

修改 .config 文件中的证书配置：

```
CONFIG_SYSTEM_TRUSTED_KEYS=""
CONFIG_SYSTEM_REVOCATION_KEYS=""
```

编译内核：

```
sudo make bzImage -j4 SKIP_STACK_VALIDATION=1
```

编译并安装内核模块：

```
sudo make modules -j4
sudo make modules_install
```

安装新内核并更新 GRUB：

```
sudo make install
sudo update-grub
sudo reboot
```

系统重启后，在 GRUB 菜单中选择：

```
Advanced options for Ubuntu
Ubuntu, with Linux 5.10.1
```

进入系统后，执行以下命令验证新内核版本：

uname -a

```
vboxuser@Ubuntu:~$ uname -a
Linux Ubuntu 5.10.1 #2 SMP Thu May 28 17:19:20 UTC 2026 x86_64 x86_64 x8
6_64 GNU/Linux
```

6.2.2 源程序清单

1. 内核系统调用函数代码

文件位置：

/usr/src/linux-5.10.1/kernel/sys.c

添加代码：

```
SYSCALL_DEFINE1(mysyscall, unsigned long long, a)
{
    return a;
}
```

2. 系统调用声明代码

文件位置：

/usr/src/linux-5.10.1/include/linux/syscalls.h

添加代码：

```
asmlinkage long sys_mysyscall(unsigned long long a);
```

3. 系统调用表代码

文件位置：

/usr/src/linux-5.10.1/arch/x86/entry/syscalls/syscall_64.tbl

添加代码：

```
335 common mysyscall sys_mysyscall
```

4. 用户空间测试程序代码

文件名：

test5.c

完整代码：

```
#include <linux/kernel.h>
#include <sys/syscall.h>
#include <unistd.h>
#include <stdio.h>
int main()
{
```

```
    printf("Your student number is %ld\n", syscall(335, 2024112639));
    return 0;
}
```

6.2.3 运行结果及解释

1. 新内核版本验证

进入 Linux 5.10.1 内核后，执行命令：

uname -a

运行结果：

Linux Ubuntu 5.10.1 #2 SMP Thu May 28 17:19:20 UTC 2026 x86_64 x86_64 x86_64 GNU/Linux

结果表明，系统已成功启动新编译安装的 Linux 5.10.1 内核。

```
vboxuser@Ubuntu:~$ uname -a
Linux Ubuntu 5.10.1 #2 SMP Thu May 28 17:19:20 UTC 2026 x86_64 x86_64 x8
6_64 GNU/Linux
```

2. 用户程序编译与运行

进入测试程序目录：

cd ~/OSLab/exp6

编译测试程序：

gcc test5.c -o test5

运行测试程序：

./test5

运行结果：

Your student number is 2024112639

```
vboxuser@Ubuntu:~/OSLab/exp6$ gcc test5.c -o test5
./test5
Your student number is 2024112639
```

3. 结果解释

测试程序通过 syscall(335, 2024112639) 发起系统调用。系统调用号 335 在系统调用表中对应 sys_mysyscall，程序执行时进入内核中的 mysyscall() 函数。

mysyscall() 接收参数 2024112639，并将该参数作为返回值返回给用户程序。运行结果与传入参数一致，说明系统调用号设置正确，内核函数注册成功，用户空间程序能够正常调用新增系统调用。

6.3 实验总结

通过本次实验，我完成了 Linux 内核系统调用的添加、编译、安装和测试过程。实验中首先查看了原系统内核版本，然后下载并解压 Linux 5.10.1 内核源码，在 kernel/sys.c 文件中添加了新的系统调用函数 mysyscall()，在 include/linux/syscalls.h 中添加了函数声明，并在 arch/x86/entry/syscalls/syscall_64.tbl 中为该系统调用设置调用号 335。

在内核修改完成后，对 Linux 5.10.1 内核进行了编译、模块安装、内核安装和 GRUB 更新。重启系统后，通过 GRUB 菜单进入新编译的 Linux 5.10.1 内核，并使用 uname -a 验证新内核启动成功。

最后，在用户空间编写测试程序 test5.c，使用 syscall(335, 2024112639) 调用新增系统调用。程序成功输出学号 2024112639，说明新增系统调用能够被用户程序正常调用，系统调用函数能够正确接收参数并返回结果。

本次实验加深了我对 Linux 系统调用机制的理解，也熟悉了内核源码修改、内核编译安装以及用户空间调用内核函数的基本流程。

实验 7　系统缺页次数统计

7.1 实验目的

理解内存管理中缺页的概念
综合运用实验 1 中/proc/文件系统， 实验 6 中系统调用、内核编译的知识；
掌握向/proc/文件系统中增加新文件的方法
掌握 Linux 内核模块的概念和操作方法

7.2 实验内容

通过在内核中添加自建变量 pfcount，统计系统从开机到当前时间的缺页次数。

- 修改 arch/x86/mm/fault.c
- 修改 include/linux/mm.h
- 修改 kernel/kallsyms.c
- 编译内核

在用户空间编写 C 语言程序，利用/proc 文件系统作为中介，获得内核中的 pfcount 的值

7.2.1 设计思想及算法流程

1. 设计思想

Linux 系统在访问尚未映射到物理内存的页面时，会触发缺页异常。缺页异常进入内核后，由缺页处理相关函数进行处理。因此，只要在缺页处理路径中加入计数变量，每发生一次缺页，就对该变量进行自增操作，即可统计系统从启动到当前时间的累计缺页次数。

本实验定义全局变量：

unsigned long volatile pfcount = 0;

变量 pfcount 用于保存系统缺页次数。在缺页处理函数中执行：

pfcount++;

每发生一次缺页，变量值增加 1。

由于用户空间程序不能直接访问内核变量，本实验通过 /proc 文件系统作为内核空间与用户空间之间的通信接口。内核模块加载后，在 /proc 目录下创建 pfcount 文件。用户执行：

cat /proc/pfcount

时，内核模块中的读取函数被调用，并将内核变量 pfcount 的当前值输出到用户空间。

2. 算法流程

开始

查看当前用户名、时间和内核版本

进入 Linux 5.10.1 内核源码目录

修改 arch/x86/mm/fault.c

定义全局变量 pfcount

在缺页处理函数中加入 pfcount++

修改 include/linux/mm.h

添加 pfcount 的 extern 声明

修改 kernel/kallsyms.c

使用 EXPORT_SYMBOL_GPL(pfcount) 导出 pfcount 符号

重新编译内核

编译内核模块

安装新内核

更新 GRUB

重启系统并进入新内核

编写 test7.c 内核模块

引用外部变量 pfcount

创建 /proc/pfcount 文件

定义读取函数输出 pfcount 的值

定义模块加载函数和卸载函数

编写 Makefile

执行 make 编译 test7.ko

使用 insmod 加载 test7.ko

使用 lsmod 检查模块是否加载成功

使用 cat /proc/pfcount 读取系统缺页次数

执行文件遍历操作

再次读取 /proc/pfcount

比较两次 pfcount 数值变化

结束

7.2.2 源程序清单

1. arch/x86/mm/fault.c 中添加的代码

在文件中添加缺页计数变量：

```
unsigned long volatile pfcount = 0;
```

在缺页处理函数中添加计数语句：

```
pfcount++;
```

2. include/linux/mm.h 中添加的代码

```
extern unsigned long volatile pfcount;
```

3. kernel/kallsyms.c 中添加的代码

```
EXPORT_SYMBOL_GPL(pfcount);
```

4. test7.c 源程序

```
#include <linux/module.h>
#include <linux/kernel.h>
#include <linux/init.h>
#include <linux/proc_fs.h>
#include <linux/seq_file.h>
#include <linux/mm.h>

extern unsigned long volatile pfcount;

static int pfcount_show(struct seq_file *m, void *v)
{
    seq_printf(m, "%lu\n", pfcount);
    return 0;
}

static int pfcount_open(struct inode *inode, struct file *file)
{
    return single_open(file, pfcount_show, NULL);
```

```
}

static const struct proc_ops pfcount_proc_ops = {
	.proc_open = pfcount_open,
	.proc_read = seq_read,
	.proc_lseek = seq_lseek,
	.proc_release = single_release,
};

static int __init test7_init(void)
{
	proc_create("pfcount", 0444, NULL, &pfcount_proc_ops);
	printk(KERN_INFO "pfcount module loaded\n");
	return 0;
}

static void __exit test7_exit(void)
{
	remove_proc_entry("pfcount", NULL);
	printk(KERN_INFO "pfcount module unloaded\n");
}

module_init(test7_init);
module_exit(test7_exit);

MODULE_LICENSE("GPL");
MODULE_AUTHOR("2024112639");
MODULE_DESCRIPTION("Page fault count proc module");
```

5. Makefile 源程序

```
obj-m += test7.o

KDIR := /lib/modules/$(shell uname -r)/build
PWD := $(shell pwd)
```

```
all:
    make -C $(KDIR) M=$(PWD) modules

clean:
    make -C $(KDIR) M=$(PWD) clean
```

7.2.3 运行结果及解释

1. 内核文件修改结果

在内核源码目录下执行命令：

grep -n "pfcount" arch/x86/mm/fault.c include/linux/mm.h kernel/kallsyms.c

运行结果如下：

arch/x86/mm/fault.c:36:unsigned long volatile pfcount = 0;

arch/x86/mm/fault.c:1446: pfcount++;

include/linux/mm.h:35:extern unsigned long volatile pfcount;

kernel/kallsyms.c:753:EXPORT_SYMBOL_GPL(pfcount);

结果表明，pfcount 已经在 fault.c 中定义并累加，在 mm.h 中声明为全局变量，并在 kallsyms.c 中完成符号导出。

```
vboxuser@Ubuntu:/usr/src/linux-5.10.1$ cd /usr/src/linux-5.10.1
grep -n "pfcount" arch/x86/mm/fault.c include/linux/mm.h kernel/kallsyms.c
whoami
date
arch/x86/mm/fault.c:36:unsigned long volatile pfcount = 0;
arch/x86/mm/fault.c:1446:         pfcount++;
include/linux/mm.h:35:extern unsigned long volatile pfcount;
kernel/kallsyms.c:752:EXPORT_SYMBOL_GPL(pfcount);
vboxuser
Mon Jun 15 08:20:06 AM UTC 2026
```

2. 内核编译与安装过程

进入内核源码目录：

cd /usr/src/linux-5.10.1

编译内核：

sudo make bzImage -j4 SKIP_STACK_VALIDATION=1

编译内核模块：

sudo make modules -j4

安装内核模块：

sudo make modules_install

安装新内核并更新 GRUB：

sudo make install

sudo update-grub

sudo reboot

重启后进入 Linux 5.10.1 内核，执行：

whoami

date

uname -a

运行结果如下：

vboxuser

Wed Jun 17 08:13:19 AM UTC 2026

Linux Ubuntu 5.10.1 #3 SMP Mon Jun 15 14:34:12 UTC 2026 x86_64 x86_64 x86_64 GNU/Linux

结果表明，系统已经进入修改后的 Linux 5.10.1 内核。

```
vboxuser@Ubuntu:~$ whoami
date
uname -a
vboxuser
Wed Jun 17 08:13:19 AM UTC 2026
Linux Ubuntu 5.10.1 #3 SMP Mon Jun 15 14:34:12 UTC 2026 x86_64 x86_64 x8
6_64 GNU/Linux
```

3. pfcount 符号导出结果

在内核源码目录下执行命令：

grep "pfcount" Module.symvers

运行结果如下：

0x129512ae pfcount vmlinux EXPORT_SYMBOL_GPL

结果表明，pfcount 已经被成功导出，外部内核模块可以访问该变量。

```
vboxuser@Ubuntu:/usr/src/linux-5.10.1$ cd /usr/src/linux-5.10.1
grep "pfcount" Module.symvers
0x129512ae      pfcount vmlinux EXPORT_SYMBOL_GPL
```

4. 模块编译结果

进入实验目录：

cd ~/OSLab/exp7

执行模块编译：

make clean

make

echo $?

运行结果中出现：

CC [M] /home/vboxuser/OSLab/exp7/test7.o

MODPOST /home/vboxuser/OSLab/exp7/Module.symvers

CC [M] /home/vboxuser/OSLab/exp7/test7.mod.o

LD [M] /home/vboxuser/OSLab/exp7/test7.ko

0

结果表明，test7.c 编译成功，并生成内核模块文件 test7.ko。

```
vboxuser@Ubuntu:~/OSLab/exp7$ cd ~/OSLab/exp7
make clean
make
echo $?
make -C /lib/modules/5.10.1/build M=/home/vboxuser/OSLab/exp7 clean
make[1]: Entering directory '/usr/src/linux-5.10.1'
  CLEAN   /home/vboxuser/OSLab/exp7/Module.symvers
make[1]: Leaving directory '/usr/src/linux-5.10.1'
make -C /lib/modules/5.10.1/build M=/home/vboxuser/OSLab/exp7 modules
make[1]: Entering directory '/usr/src/linux-5.10.1'
  CC [M]  /home/vboxuser/OSLab/exp7/test7.o
  MODPOST /home/vboxuser/OSLab/exp7/Module.symvers
  CC [M]  /home/vboxuser/OSLab/exp7/test7.mod.o
  LD [M]  /home/vboxuser/OSLab/exp7/test7.ko
make[1]: Leaving directory '/usr/src/linux-5.10.1'
0
```

5. 模块加载与 /proc/pfcount 读取结果

加载模块：

sudo insmod test7.ko

检查模块是否加载成功：

lsmod | grep test7

运行结果如下：

test7 16384 0

读取 /proc/pfcount：

whoami

date

cat /proc/pfcount

运行结果如下：

vboxuser

Wed Jun 17 09:24:15 AM UTC 2026

46151824

结果表明，test7.ko 模块已经成功加载，/proc/pfcount 文件已经创建成功，用户空间可以通过读取 /proc/pfcount 获取内核变量 pfcount 的值。

```
vboxuser@Ubuntu:~/OSLab/exp7$ sudo insmod test7.ko
lsmod | grep test7
whoami
date
cat /proc/pfcount
[sudo] password for vboxuser:
test7                  16384  0
vboxuser
Wed Jun 17 09:24:15 AM UTC 2026
46151824
```

6. 缺页次数变化验证

继续执行：

cat /proc/pfcount

ls -R /usr/src/linux-5.10.1 >/dev/null

cat /proc/pfcount

运行结果如下：

46153607

46154808

结果显示，执行文件遍历操作后，pfcount 的值由 46153607 增加到 46154808。说明系统运行过程中仍然在发生缺页，pfcount 能够持续统计系统从开机到当前时间的缺页次数。

```
vboxuser@Ubuntu:~/OSLab/exp7$ cat /proc/pfcount
ls -R /usr/src/linux-5.10.1 >/dev/null
cat /proc/pfcount
46153607
46154808
```

7. 结果解释

pfcount 是在内核缺页处理路径中定义并累加的全局变量。每次系统发生缺页异常时，缺页处理函数被执行，pfcount 增加 1。

test7.ko 模块加载后，在 /proc 文件系统中创建 /proc/pfcount 文件。用户执行 cat /proc/pfcount 时，模块中的读取函数被调用，并将内核变量 pfcount 的当前值输出到用户空间。

实验结果中，/proc/pfcount 能够输出数值，并且在执行文件遍历操作后数值继续增加。该结果说明缺页统计功能正常，/proc 文件系统能够作为内核空间与用户空间之间的通信接口使用。

7.3 实验总结

本实验完成了系统缺页次数统计功能的设计、内核修改、内核编译、模块编写、模块加载和结果验证。

在内核修改部分，实验通过修改 arch/x86/mm/fault.c 文件定义缺页计数变量 pfcount，并在缺页处理函数中对其进行累加；通过修改 include/linux/mm.h 文件声明该全局变量；通过修改 kernel/kallsyms.c 文件导出 pfcount 符号，使外部内核模块能够访问该变量。

在用户空间访问部分，实验编写了 test7.c 内核模块，并通过 /proc 文件系统创建 /proc/pfcount 文件。模块加载后，用户空间可以使用 cat /proc/pfcount 读取系统从开机到当前时间的缺页次数。

实验结果显示，test7.ko 模块能够成功编译并加载，/proc/pfcount 文件能够正常输出缺页次数。在执行文件遍历操作后，pfcount 数值继续增加，说明系统缺页统计功能实现正确，内核变量能够通过 /proc 文件系统传递到用户空间。

通过本实验，进一步理解了 Linux 缺页异常处理过程，掌握了在内核中添加自定义统计变量的方法，也熟悉了 Linux 内核模块和 /proc 文件系统在内核态与用户态通信中的作用。